

PRACTICAL-1

AIM: To implement and analyze the time complexity of different sorting algorithms, namely Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort, and compare their performance in different cases.

BUBBLE SORT:

ALGORITHM:

Bubble Sort Algorithm

Input : Array A (with n elements)
Output: Sorted array A in ascending order
Step 1: Start
Step 2: Read n and array A
Step 3: Set $i = 0$
Step 4: Repeat while $i < n - 1$
 * Set $j = 0$
 * Compare $A[j]$ and $A[j+1]$
 * If $A[j] > A[j+1]$ swap them
 * Increment j until $j < n - i - 1$
 * Increment $i = i + 1$
Step 5: Display sorted array A
Step 6: Stop

PROGRAM/CODE:

```
#include <iostream>
using namespace std;

class bubblesort
{
public:
    int size, arr[100], i;

    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;

        cout << "Enter array elements:\n";
        for (i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }

    void printarray()
    {
        for (i = 0; i < size; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    void bubblelogic()
    {
        int j, temp;

        for (i = 0; i < size - 1; i++)
        {
            for (j = 0; j < size - i - 1; j++)
            {
                if (arr[j] > arr[j + 1])
                {
                    temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
}
```

```
    }  
  }  
}  
};  
  
int main()  
{  
    bubblesort bs;  
  
    bs.arrayinput();  
  
    cout << "Before sorting: ";  
    bs.printarray();  
  
    bs.bubblelogic();  
  
    cout << "After sorting: ";  
    bs.printarray();  
  
    return 0;  
}
```

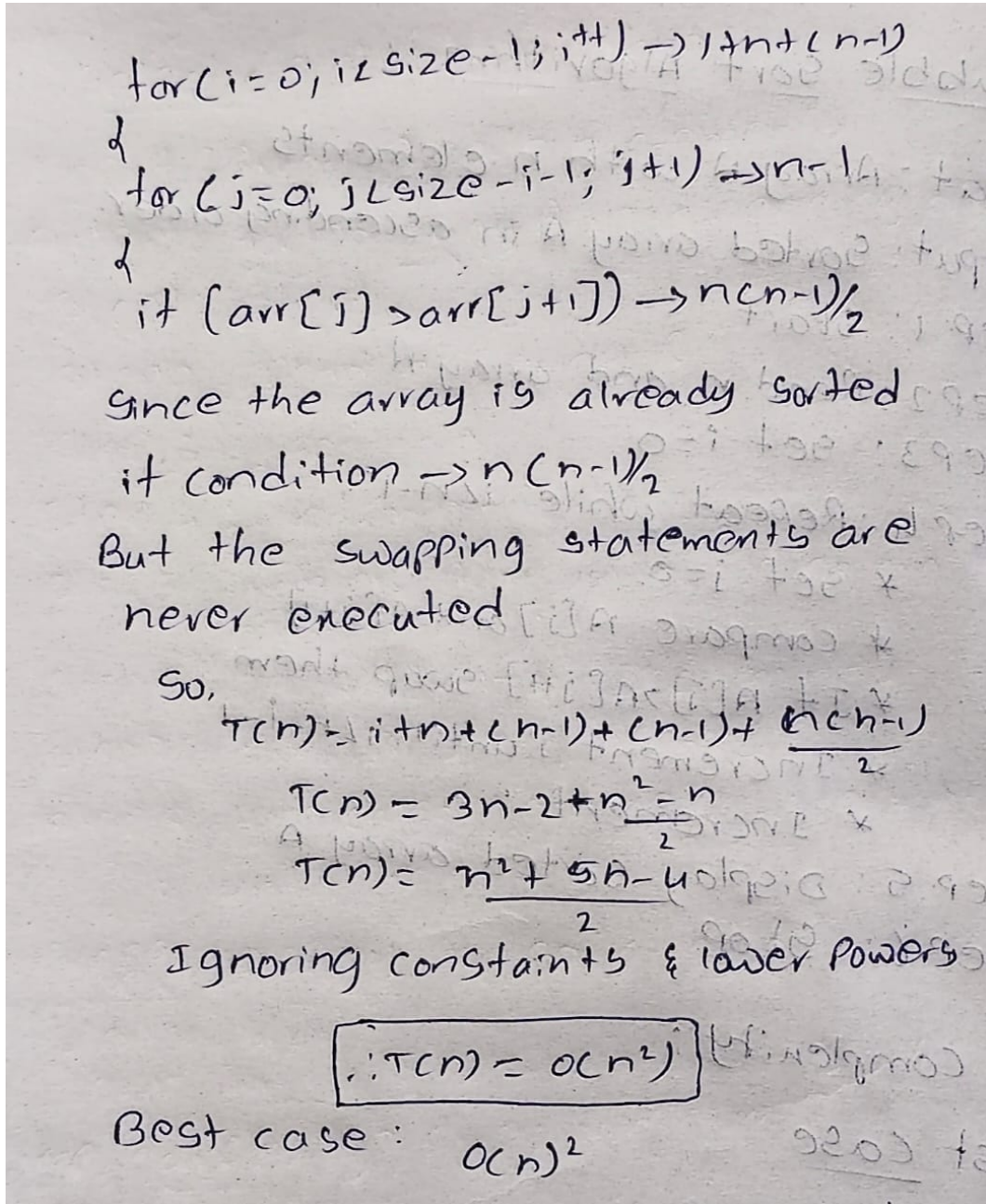
OUTPUT :

```
Enter array size: 5  
Enter array elements:  
23  
44  
54  
37  
77  
Before sorting: 23 44 54 37 77  
After sorting: 23 37 44 54 77
```

TIME COMPLEXITY:

1. BEST CASE: Example: 1 2 3 4 5

Even though the array is already sorted, your code still performs all comparisons.



$\text{for}(i=0; i < \text{size}-1; i++) \rightarrow 1+n+(n-1)$
 $\{$
 $\text{for}(j=0; j < \text{size}-i-1; j++) \rightarrow n-1$
 $\{$
 $\text{if}(\text{arr}[j] > \text{arr}[j+1]) \rightarrow n(n-1)/2$
 Since the array is already sorted
 if condition $\rightarrow n(n-1)/2$
 But the swapping statements are
 never executed
 So,
 $T(n) = 1+n+(n-1) + (n-1) + \frac{n(n-1)}{2}$
 $T(n) = 3n-2 + \frac{n^2-n}{2}$
 $T(n) = \frac{n^2 + 5n - 4}{2}$
 Ignoring constants & lower powers
 $\therefore T(n) = O(n^2)$
 Best case : $O(n)^2$

2. AVERAGE CASE:

Example: 5 2 8 1 6

Some comparisons result in swapping and some don't.

The nested loops still perform:

$$n(n-1)/2$$

comparisons.

On average, approximately half of the comparisons cause a swap

$$\frac{1}{2} \times \frac{n(n-1)}{2} = \frac{n(n-1)}{4}$$

Each swap has 3 statements:

$$\begin{aligned} \text{temp} = \text{arr}[j] &\rightarrow n(n-1)/4 \\ \text{arr}[j] = \text{arr}[j+1] &\rightarrow n(n-1)/4 \\ \text{arr}[j+1] = \text{temp} &\rightarrow n(n-1)/4 \end{aligned}$$

So,

$$T(n) = 1 + n + (n-1) + (n-1) + \frac{n(n-1)}{2} + \frac{3n(n-1)}{4}$$

$$T(n) = 3n - 2 + \frac{n(n-1)}{2} + \frac{3n(n-1)}{4}$$

Taking the highest order term:

$$T(n) = O(n)^2$$

Average case: $O(n^2)$

3. WORST CASE: Example: 5 4 3 2 1

Every comparison causes a swap.

Number of comparisons: $n(n-1)/2$

And every comparison performs all 3 swap statements.

Worst case:

$$temp = arr[j] \rightarrow \frac{n(n-1)}{2}$$

$$arr[j] = arr[j+1] \rightarrow \frac{n(n-1)}{2}$$

$$arr[j+1] = temp \rightarrow \frac{n(n-1)}{2}$$

Therefore,

$$T(n) = 1 + n + (n-1) + (n-1) + \frac{4n(n-1)}{2}$$

$$T(n) = 3n - 2 + 2n(n-1)$$

$$T(n) = 2n^2 + n - 2$$

$$T(n) = O(n)^2$$

Worst case : $O(n)^2$

SELECTION SORT:

ALGORITHM:

Selection Sort Algorithm:

Input: Array A with n elements
Output: Sorted array A in ascending order

Step 1: Start

Step 2: Read n & array A

Step 3: set $i = 0$

Step 4: set $min = i$

Step 5: Compare $A[min]$ with the remaining elements

Step 6: If $A[j] < A[min]$, set $min = j$

Step 7: swap $A[i]$ and $A[min]$

Step 8: Increment i and repeat steps 4-7 until $i = n-1$

Step 9: Display sorted array A

Step 10: Stop

PROGRAM/CODE :

```
#include <iostream>
using namespace std;

class SelectionSort
{
public:
    int size, arr[100], i, j, temp;

    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;

        cout << "Enter array elements: ";
        for (i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }

    void printarray()
    {
        for (i = 0; i < size; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    void selectionsortlogic()
    {
        for (i = 0; i < size - 1; i++)
        {
            int min = i;

            for (j = i + 1; j < size; j++)
            {
                if (arr[j] < arr[min])
                {
                    min = j;
                }
            }

            if (min != i)
```

```
{
    temp = arr[i];
    arr[i] = arr[min];
    arr[min] = temp;
}
}
};

int main()
{
    SelectionSort ss;

    ss.arrayinput();

    cout << "Before sorting: ";
    ss.printarray();

    ss.selectionsortlogic();

    cout << "After sorting: ";
    ss.printarray();

    return 0;
}
```

OUTPUT :

```
Enter array size: 5
Enter array elements: 23
11
45
43
32
Before sorting: 23 11 45 43 32
After sorting: 11 23 32 43 45
```

TIME COMPLEXITY:

1. BEST CASE:

Example: 1 2 3 4 5

Even though the array is sorted, Selection Sort still checks every element to find the minimum.

Best Case:
order loop
for $i=0; i < n-1; i \rightarrow 1 + n + (n-1)$
Inner loop
The total no of comparisons is
 $(n-1) + (n-2) + (n-3) + \dots + 1$
 $= \frac{n(n-1)}{2}$
so,

if $(arr[i] < arr[min]) \rightarrow n(n-1)/2$
since the array is already sorted $min=i$
every time but the swap statement still
execute in the normal code
 $temp = arr[i] \rightarrow n-1$
 $arr[i] = arr[min] \rightarrow n-1$
 $arr[min] = temp \rightarrow n-1$
therefore,
 $T(n) = 1 + n + (n-1) + \frac{n(n-1)}{2} + 3(n-1)$
 $T(n) = 5n - 3 + \frac{n(n-1)}{2}$
 $T(n) = 5n - 3 + \frac{n^2 - n}{2}$
 $\therefore T(n) = O(n^2)$
Best case $O(n^2)$

2. AVERAGE CASE:

For a randomly arranged array, Selection Sort still performs exactly the same number of comparisons.

Inner loop

$$n(n-1)/2$$

comparisons.

The `min = j` statement executes whenever a new minimum is found. Its exact average count is less important for Big-O.

The three swap statements execute $(n-1)$ times:

Handwritten derivation of the average case time complexity for Selection Sort:

$$\begin{aligned}
 & \text{temp} = \text{arr}[i] \rightarrow n-1 \\
 & \text{arr}[i] = \text{arr}[\text{min}] \rightarrow n-1 \\
 & \text{arr}[\text{min}] = \text{temp} \rightarrow n-2 \\
 & \text{Therefore the dominant part is} \\
 & T(n) = \frac{n(n-1)}{2} + 3(n-1) + 1 + n + 2(n-1) \\
 & \boxed{T(n) = O(n^2)} \\
 & \text{Avg case: } O(n^2)
 \end{aligned}$$

3. WORST CASE:

Example: 5 4 3 2 1

Selection Sort again performs the same comparisons:

Handwritten calculation of the number of comparisons in the worst case for Selection Sort:

$$\begin{aligned}
 & (n-1) + (n-2) + (n-3) + \dots \\
 & = \frac{n(n-1)}{2}
 \end{aligned}$$

The min assignment can happen many times, but the dominant term is still (n^2) .
Swapping:

$$\begin{aligned}
 & \text{temp} = \text{arr}[i] \rightarrow n-1 \\
 & \text{arr}[i] = \text{arr}[\text{min}] \rightarrow n-1 \\
 & \text{arr}[\text{min}] = \text{temp} \rightarrow n-1 \\
 & \text{Therefore} \\
 & T(n) = 1 + n + (n-1) + (n-1) + \frac{n(n-1)}{2} + 4(n-1) \\
 & T(n) = 6n - 4 + \frac{n^2 - n}{2} \\
 & T(n) = 6n - 4 + \frac{n^2}{2} - \frac{n}{2} \\
 & T(n) = n^2 + 11n - 8\frac{1}{2} \\
 & \text{worst case : } O(n^2)
 \end{aligned}$$

INSERTION SORT:

ALGORITHM:

Insertion Sort

Input: Array A with n elements
output: sorted array A in ascending order

Step 1: start
Step 2: Read n and Array A
Step 3: set $i = 1$
Step 4: set $\text{Key} = A[i]$ & $j = i - 1$
Step 5: while $j > 0$ & $A[j] > \text{Key}$, shift $A[j]$ to $A[j+1]$
Step 6: Insert Key at $A[j+1]$
Step 7: Increment i & repeat step 4-6 until $i \leq n$
Step 8: display sorted array A
Step 9: stop

PROGRAM/CODE:

```
#include <iostream>
using namespace std;

class InsertionSort
{
public:
    int size, arr[100], i, j, temp;

    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;

        cout << "Enter array elements: ";
        for (i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }

    void printarray()
    {
        for (i = 0; i < size; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    void insertionsortlogic()
    {
        for (i = 1; i < size; i++)
        {
            temp = arr[i];
            j = i - 1;

            while (j >= 0 && arr[j] > temp)
            {
                arr[j + 1] = arr[j];
                j--;
            }

            arr[j + 1] = temp;
        }
    }
}
```

```
}  
};  
  
int main()  
{  
    InsertionSort is;  
  
    is.arrayinput();  
  
    cout << "Before sorting: ";  
    is.printarray();  
  
    is.insertionsortlogic();  
  
    cout << "After sorting: ";  
    is.printarray();  
  
    return 0;  
}
```

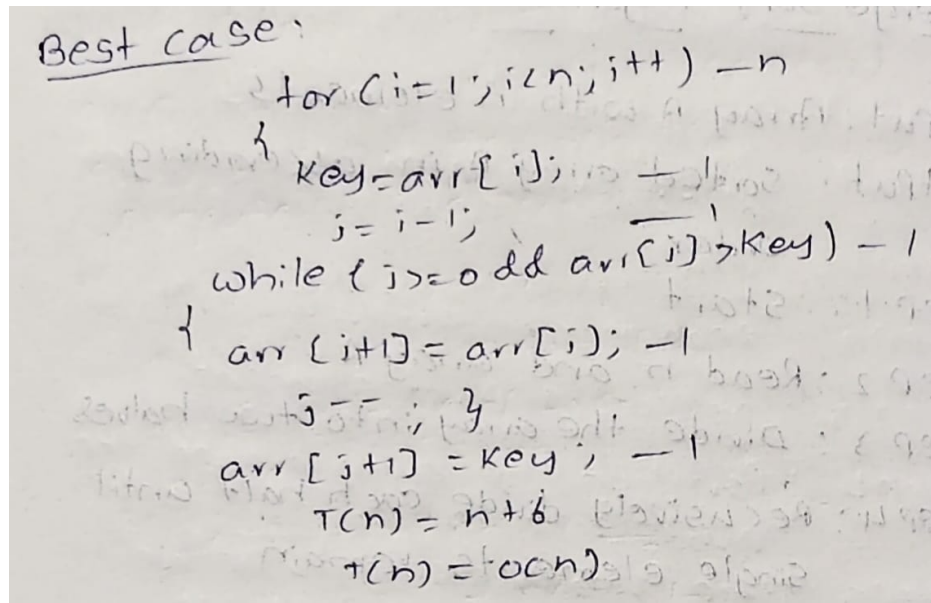
OUTPUT:

```
Enter array size: 5  
Enter array elements: 44  
65  
45  
21  
12  
Before sorting: 44 65 45 21 12  
After sorting: 12 21 44 45 65
```


TIME COMPLEXITY :

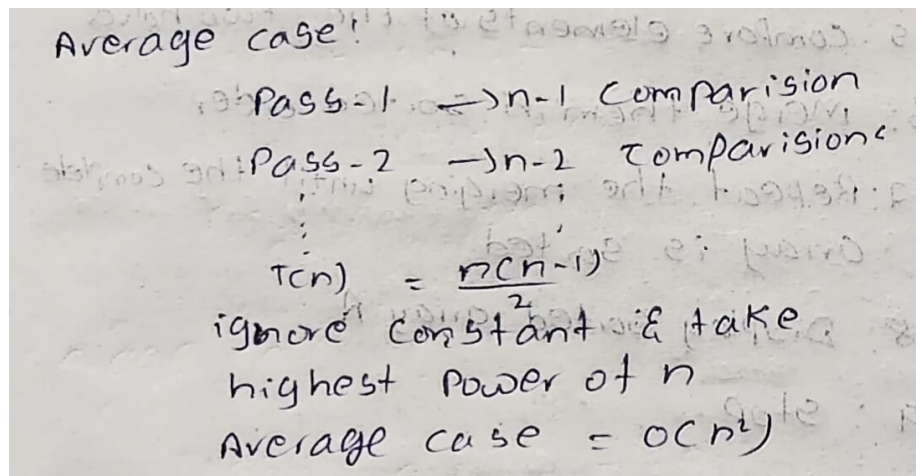
1.BEST CASE:

The best case occurs when the array is already sorted, so each element requires only one comparison.



2.AVERAGE CASE:

The average case occurs when the elements are randomly arranged, so each element requires approximately half of the comparisons.



3.WORST CASE:

The worst case occurs when the array is in reverse order, so each element requires the maximum number of comparisons.

worst case:

The worst case occurs when pivot is always smallest or largest

$$n + (n-1) + (n-2) + \dots + 1$$
$$T(n) = \frac{n(n+1)}{2}$$

worst case = $O(n)^2$

MERGE SORT:

ALGORITHM:

Merge Sort Algorithm:

Input: Array A with n elements
Output: Sorted array A in ascending order

Step 1: Start

Step 2: Read n and array A

Step 3: Divide the array into two halves

Step 4: Recursively divide each half until single elements remain

Step 5: Compare elements of the two halves

Step 6: Merge them in sorted order

Step 7: Repeat the merging until the complete array is sorted.

Step 8: Display sorted array A

Step 9: Stop

PROGRAM/CODE :

```
#include <iostream>
using namespace std;

class MergeSort
{
public:
    int size, arr[100], i;

    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;

        cout << "Enter array elements:\n";
        for (i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }

    void printarray()
    {
        for (i = 0; i < size; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    void merge(int low, int mid, int high)
    {
        int temp[100];
        int i = low;
        int j = mid + 1;
        int k = low;

        while (i <= mid && j <= high)
        {
            if (arr[i] <= arr[j])
            {
                temp[k] = arr[i];
                i++;
            }
            else
```

```
{  
    temp[k] = arr[j];  
    j++;  
}  
k++;  
}
```

```
while (i <= mid)  
{  
    temp[k] = arr[i];  
    i++;  
    k++;  
}
```

```
while (j <= high)  
{  
    temp[k] = arr[j];  
    j++;  
    k++;  
}
```

```
for (i = low; i <= high; i++)  
{  
    arr[i] = temp[i];  
}  
}
```

```
void mergellogic(int low, int high)  
{  
    if (low < high)  
    {  
        int mid = (low + high) / 2;  
  
        mergellogic(low, mid);  
        mergellogic(mid + 1, high);  
  
        merge(low, mid, high);  
    }  
}  
};
```

```
int main()  
{  
    MergeSort ms;
```

```
ms.arrayinput();

cout << "Before sorting: ";
ms.printarray();

ms.mergellogic(0, ms.size - 1);

cout << "After sorting: ";
ms.printarray();

return 0;
}
```

OUTPUT:

```
Enter array size: 5
Enter array elements:
43
45
45
65
67
Before sorting: 43 45 45 65 67
After sorting: 43 45 45 65 67
```

TIME COMPLEXITY:

1. BEST CASE:

Even if the array is already sorted, Merge Sort still divides the array and performs the merge operation.

2. AVERAGE CASE:

Example: 5 2 8 1 6

The array is randomly arranged, but Merge Sort still divides the array into two halves and merges them.

3. WORST CASE:

The worst case occurs when the elements require the maximum number of comparisons during the merge operation.

However, Merge Sort still divides the array into two halves.

worst case :

```

merge sort (arr, low, high) → T(n/2)
{
  if (low < high) → n
  {
    mid = (low + high) / 2 ; → 1
    merge sort (arr, low, mid);
    merge sort (arr, mid+1, high) → T(n/2)
    merge sort (arr, low, mid, high); → n
  }
}

```

therefore,

$$T(n) = 1 + 1 + T(n/2) + T(n/2) + n$$

$$= 2T(n/2) + n + 2$$

$$T(n) = 2T(n/2) + n$$

using master theorem

$$a = 2 \quad b = 2 \quad f(n) = n$$

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

therefore $T(n) = \Theta(n \log n)$

QUICK SORT:

ALGORITHM:

Quick Sort Algorithm:

Input: Array A with 'n' elements
Output: sorted array A in ascending order

Step 1 : Start
Step 2 : Read n and array A
Step 3 : select a pivot element
Step 4 : Partition the array around the pivot
Step 5 : Place smaller elements before the pivot & larger after it
Step 6 : Recursively apply Quick Sort to left & right part
Step 7 : Repeat until all elements are sorted
Step 8 : Display sorted array A
Step 9 : stop

PROGRAM/CODE :

```
#include <iostream>
#include <utility>
using namespace std;

class QuickSort
{
public:
    int size, arr[100];

    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;

        cout << "Enter array elements: ";
        for (int i = 0; i < size; i++)
        {
```

```
        cin >> arr[i];
    }
}

void printarray()
{
    for (int i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int partition(int lb, int ub)
{
    int pivot = arr[lb];
    int start = lb;
    int end = ub;

    while (start < end)
    {
        while (start <= ub && arr[start] <= pivot)
            start++;

        while (arr[end] > pivot)
            end--;

        if (start < end)
            swap(arr[start], arr[end]);
    }

    swap(arr[lb], arr[end]);

    return end;
}

void quicksortlogic(int lb, int ub)
{
    if (lb < ub)
    {
        int loc = partition(lb, ub);

        quicksortlogic(lb, loc - 1);
        quicksortlogic(loc + 1, ub);
    }
}
```

```
}  
  
void sort()  
{  
    quicksortlogic(0, size - 1);  
}  
};  
  
int main()  
{  
    QuickSort qs;  
  
    qs.arrayinput();  
  
    cout << "Before sorting: ";  
    qs.printarray();  
  
    qs.sort();  
  
    cout << "After sorting: ";  
    qs.printarray();  
  
    return 0;  
}
```

OUTPUT :

```
Enter array size: 5  
Enter array elements: 22  
32  
12  
14  
19  
Before sorting: 22 32 12 14 19  
After sorting: 12 14 19 22 32
```

TIME COMPLEXITY:

1.BEST CASE:

The best case occurs when the pivot divides the array into two nearly equal parts.

2.AVERAGE CASE:

The average case occurs when the pivot divides the array into reasonably balanced parts on average.

3.WORST CASE:

The worst case occurs when the pivot is always the smallest or largest element, creating one subarray of size $(n-1)$ and the other of size 0.

Time Complexity

```

Partition (arr, lb, ub)
{
    pivot = arr[lb] → 1
    start = lb; → 1
    end = ub; → 1
    while (start < end) → n
    {
        while (start <= ub && arr[start] <= pivot) → n
        {
            start = start + 1; → n
        }
        while (arr[end] > pivot) → n
        {
            end = end - 1; → n
        }
        if (start < end) → n
        {
            swap (arr[start], arr[end]); → n
        }
        swap (arr[lb], arr[end]); → 1
        return end; → 1
    }
}

Quick Sort (arr, lb, ub)
{
    if (lb < ub) → 1
    {
        loc = Partition(arr, lb, ub); → n
        Quick Sort (arr, lb, loc-1); → T(n)
        Quick Sort (arr, loc+1, ub); → T(n-k-1)
    }
}
    
```


for one partition

$$T(n) = 1 + 1 + 1 + n + n + n + n + n + n + 1 + 1$$

$$= 7n + 5$$

so one partition: $O(n)$

Best / Average case: $T(n) = 2T(n/2) + 7n + 5$

$$T(n) = O(n \log n)$$

worst case: $T(n) = T(n-1) + 7n + 5$

$$T(n) = \sum_{i=1}^n (7i + 5) = \frac{7n(n+1)}{2} + 5n$$

$$= \frac{7n^2 + 17n}{2}$$

$$T(n) = O(n^2)$$

Best case: $O(n \log n)$

Average case: $O(n \log n)$

worst case: $O(n^2)$

$$T(n) = T(n-1) + n$$

$$T(n) = n + (n-1) + (n-2) + \dots + 1$$

$$T(n) = \frac{n(n+1)}{2}$$

worst case: $O(n^2)$

CONCLUSION: Thus, Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort were successfully implemented and their time complexities were analyzed. The comparison helped in understanding the performance of different sorting algorithms in best, average, and worst cases.